

RESUMEN – CLASES DEL BLOQUE 4

Clase	Dir.	Uso principal
<code>ObjectOutputStream</code>	Escritura	Serializar objetos a bytes en archivo
<code>ObjectInputStream</code>	Lectura	Deserializar objetos desde archivo
<code>FileOutputStream</code>	Escritura	Envuelto por <code>ObjectOutputStream</code>
<code>FileInputStream</code>	Lectura	Envuelto por <code>ObjectInputStream</code>
<code>Serializable</code>	—	Interfaz marcadora obligatoria en la clase

MÉTODOS CLAVE

<code>oos.writeObject(obj)</code> serializa y guarda el objeto	<code>ois.readObject()</code> lee y deserializa → <code>Object</code>
<code>(Tipo) ois.readObject()</code> cast obligatorio al tipo real	<code>transient</code> campo excluye el campo de la serialización
<code>serialVersionUID</code> controla versiones de la clase	<code>EOFException</code> señal de fin al leer varios objetos

MODOS DE ACCESO A FICHEROS

Modo	Descripción	Ejemplo
Secuencial	Se lee de inicio a fin, en orden	Serialización, text files
Directo / Aleatorio	Saltar a cualquier posición del archivo	<code>RandomAccessFile</code>

FICHEROS DE TEXTO VS BINARIOS

	Texto (.txt)	Binario (.dat)
Legible	✓ Humanos	✗ Solo máquina
Eficiencia	Menor	Mayor
Usado con	<code>FileReader/Writer</code>	<code>ObjectStream</code>
Contenido	Caracteres	Bytes del objeto

✗ ERRORES COMUNES

- No implementar `Serializable` → `NotSerializableException` en runtime
- No hacer cast en `readObject()` → el método devuelve `Object`, siempre hay que castear
- Cambiar la clase tras serializar sin actualizar `serialVersionUID` → `InvalidClassException`
- No capturar `ClassNotFoundException` en el `catch` de `readObject()` → error de compilación
- Guardar varios objetos uno a uno y leer con bucle infinito → usar `EOFException` para detectar el fin, no comparar con `null`

EJEMPLO COMPLETO – GUARDAR Y LEER LISTA

Patrón más práctico: serializar una `ArrayList` entera en una sola operación de escritura/lectura.

GUARDAR ARRAYLIST

```
import java.io.*;
import java.util.ArrayList;

public class GuardarLista {
    public static void main(String[] args) {
        ArrayList<Persona> lista = new ArrayList<>();
        lista.add(new Persona("Ana", 20));
        lista.add(new Persona("Luis", 25));

        try (ObjectOutputStream oos =
            new ObjectOutputStream("personas.dat")) {
            oos.writeObject(lista); // + 1 sola llamada
        } catch (IOException e) {
            System.out.println("Error");
        }
    }
}
```

LEER ARRAYLIST

```
try (ObjectInputStream ois =
    new ObjectInputStream(
        new FileInputStream("personas.dat"))) {

    ArrayList<Persona> lista =
        (ArrayList<Persona>) ois.readObject();

    for (Persona p : lista)
        System.out.println(p.nombre);

} catch (IOException | ClassNotFoundException e) {
    System.out.println("Error");
}
```

📖 LEER VARIOS OBJETOS CON EOFEXCEPTION

Cuando se guardan objetos **uno a uno** (no en lista), se usa un bucle infinito que termina cuando `readObject()` lanza `EOFException`.

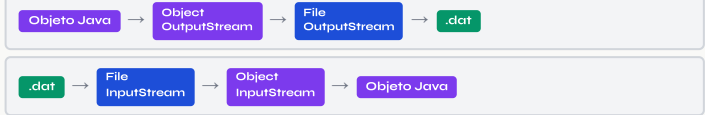
```
try {
    while (true) {
        Persona p = (Persona) ois.readObject();
        System.out.println(p.nombre);
    }
} catch (EOFException e) {
    System.out.println("Fin del archivo");
} catch (IOException | ClassNotFoundException e) {
    System.out.println("Error");
}
```

Preferible: guardar una `ArrayList` en una sola operación. Evita el bucle de `EOFException` y es más limpio.

Persistencia y Serialización

BLOQUE 4 · CHULETARIO · TRÍPTICO A4

CONCEPTO DE SERIALIZACIÓN



JERARQUÍA DEL BLOQUE

```
java.io
├─ OutputStream
│   └─ ObjectOutputStream + writeObject(obj)
│       └─ envuelve FileOutputStream
├─ InputStream
│   └─ ObjectInputStream + readObject() → Object
│       └─ envuelve FileInputStream
├─ Serializable + interfaz marcadora (java.io)
└─ EOFException + señal de fin al deserializar
```

CONTENIDOS DE ESTE TRÍPTICO

- Interior izq.** — Persistencia, `Serializable`, `ObjectOutputStream`
- Interior cent.** — `ObjectInputStream`, `transient`, `serialVersionUID`
- Interior der.** — Varios objetos, comparativa, tabla resumen
- Solapa** — Ejemplo completo con lista, `EOFException`
- Contraportada** — Tabla de clases, métodos, modos, errores comunes

LOS 4 BLOQUES DE I/O

Bloque	Clases clave	Dato
1 — Bytes	<code>FileInputStream/OutputStream</code>	byte
2 — Texto	<code>FileReader/Writer</code> + <code>Buffered*</code>	char
3 — Scanner	<code>Scanner</code> + <code>File/System.in</code>	tipos
4 — Objetos	<code>ObjectInputStream/OutputStream</code>	Object

📁 ¿QUÉ ES LA PERSISTENCIA?

Capacidad de **guardar datos de forma permanente**, más allá de la ejecución del programa. Hasta ahora guardábamos texto plano ("Juan, 25"). Con serialización guardamos **objetos completos**.

Serialización	Deserialización
Objeto Java → bytes → archivo .dat	Archivo .dat → bytes → Objeto Java

🔥 INTERFAZ SERIALIZABLE – REQUISITO OBLIGATORIO

Para serializar una clase, **debe implementar Serializable**. Es una interfaz marcadora: no exige implementar ningún método, solo declara que la clase puede serializarse.

```
import java.io.Serializable;

public class Persona implements Serializable {
    String nombre;
    int edad;
    // Recomendado: versión de la clase
    private static final long serialVersionUID = 1L;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }
}
```

✓ No hace falta implementar métodos - ✓ Solo el `implements Serializable` es suficiente - ✓ Todos los atributos también deben ser serializables (tipos primitivos y String lo son por defecto)

📁 OBJECTOUTPUTSTREAM – GUARDAR UN OBJETO

```
import java.io.*;

public class GuardarObjeto {
    public static void main(String[] args) {
        Persona p = new Persona("Gabriel", 30);

        try (ObjectOutputStream oos =
            new ObjectOutputStream(
                new FileOutputStream("persona.dat"))) {
            oos.writeObject(p);
            System.out.println("Objeto guardado");
        } catch (IOException e) {
            System.out.println("Error");
        }
    }
}
```

Código	Qué hace
<code>new ObjectOutputStream(new FileOutputStream("f"))</code>	Encadena los flujos para escribir objetos en archivo
<code>oos.writeObject(p)</code>	Convierte el objeto a bytes y lo escribe

📁 OBJECTINPUTSTREAM – LEER UN OBJETO

```
import java.io.*;

public class LeerObjeto {
    public static void main(String[] args) {
        try (ObjectInputStream ois =
            new ObjectInputStream(
                new FileInputStream("persona.dat"))) {

            Persona p = (Persona) ois.readObject();
            System.out.println(p.nombre + " - " + p.edad);

        } catch (IOException | ClassNotFoundException e) {
            System.out.println("Error");
        }
    }
}
```

Código	Qué hace
<code>new ObjectInputStream(new FileInputStream("f"))</code>	Encadena flujos para leer objetos desde archivo
<code>ois.readObject()</code>	Devuelve un <code>Object</code> — siempre hay que castear
<code>(Persona) ois.readObject()</code>	Cast al tipo real para poder usar sus campos
<code>ClassNotFoundException</code>	Si la clase <code>Persona</code> no existe en el proyecto al leer

`readObject()` lanza dos excepciones: `IOException` y `ClassNotFoundException`. El multi-catch `IOException | ClassNotFoundException` captura ambas en un solo bloque.

🔒 TRANSIENT – EXCLUIR ATRIBUTOS

La palabra clave `transient` marca un atributo para que **no se serialice**. Su valor no se guarda en el archivo y se inicializa a `null` / o al deserializar.

```
public class Usuario implements Serializable {
    String nombre;        // + si se serializa
    transient String password; // + NO se guarda
    transient int sesiones; // + NO se guarda
}
```

- Útil para **datos sensibles** (contraseñas, tokens)
- Útil para **datos calculables** que no necesitan persistir
- Al deserializar, `password` valdrá `null` y `sesiones` valdrá `0`

🔖 SERIALVERSIONUID – CONTROL DE VERSIONES

Identifica la versión de la clase. Si cambias la clase tras serializar objetos y el UID no coincide, Java lanza `InvalidClassException` al leer.

```
private static final long serialVersionUID = 1L;
// Declararlo explícitamente evita advertencias del IDE
// Incrementa el valor (2L, 3L...) al cambiar la clase
// si los archivos .dat ya no son compatibles
```

Situación	Resultado
Mismo UID, misma clase	✓ Lee sin problemas
Clase modificada, mismo UID	⚠ Puede funcionar con campos nuevos = null
UID diferente	✗ <code>InvalidClassException</code>

📁 GUARDAR VARIOS OBJETOS (UNO A UNO)

```
try (ObjectOutputStream oos =
    new ObjectOutputStream(
        new FileOutputStream("personas.dat"))) {
    oos.writeObject(new Persona("Ana", 20));
    oos.writeObject(new Persona("Luis", 25));
    oos.writeObject(new Persona("Marta", 30));
}
```

LEER CON EOFEXCEPTION

```
try (ObjectInputStream ois = ...) {
    try {
        while (true) {
            Persona p = (Persona) ois.readObject();
            System.out.println(p.nombre);
        }
    } catch (EOFException e) {
        System.out.println("Fin"); // fin esperado
    }
}
```

🔖 UN OBJETO VS LISTA – ¿CUÁNDO USAR CADA FORMA?

	Objetos uno a uno	ArrayList serializada
Escritura	<code>writeObject()</code> × N	<code>writeObject(lista)</code> × 1
Lectura	Bucle + <code>EOFException</code>	<code>readObject()</code> + cast
Código	Más verboso	Más limpio
Preferido	Streams de eventos	✓ Uso general

📁 TABLA RESUMEN DEL BLOQUE

Operación	Clase	Método
Habilitar serialización	<code>implements Serializable</code>	—
Guardar objeto	<code>ObjectOutputStream</code>	<code>writeObject(obj)</code>
Leer objeto	<code>ObjectInputStream</code>	<code>readObject()</code>
Cast al leer	—	<code>(Tipo) readObject()</code>
Excluir campo	—	<code>transient campo</code>
Versionar clase	—	<code>serialVersionUID = 1L</code>
Fin de archivo	<code>EOFException</code>	catch esperado
Guardar lista	<code>ObjectOutputStream</code>	<code>writeObject(lista)</code>
Leer lista	<code>ObjectInputStream</code>	<code>(ArrayList<T>) readObject()</code>

Regla de oro: implementa `Serializable` + declara `serialVersionUID` + usa `ArrayList` cuando tengas varios objetos + captura siempre `IOException | ClassNotFoundException`.